

Multiplicación de matrices en Python: análisis de rendimiento (transpuesta, paralelismo y Numba)

Institución: UNTDF

Carrera: Lic. en Sistemas, 5to año

Materia: Sistemas Paralelos

Docente: MsC. Federico González Brizzio

Abstract

Se midió el tiempo de la multiplicación ($C = A \cdot B$) con matrices de enteros en perfil cúbico (`--perfil cubo`, (A) y (B) de (c c)), semilla `--seed 2026` y cota `--max-val 256`, comparando el secuencial con método transpuesto y el triple bucle tradicional, paralelismo con `ThreadPoolExecutor`, `threading` y `ProcessPoolExecutor` por filas, y `numba.njit` (serial y `prange` con hilos 4/8 en el equipo de prueba). Todos los métodos compartieron el mismo checksum (suma de elementos de (A B)) y pasaron la comprobación $(A \cdot B)^T = B^T \cdot A^T$. Sobre un Intel i5-10300H (4 núcleos físicos, 8 lógicos) y Python 3.14.3 *free-threaded* (GIL deshabilitado), el secuencial transpuesto supera al tradicional por localidad; el paralelismo puro gana frente al tradicional pero sublineal al aumentar *workers*; Numba compila a código nativo y domina el ranking incluso sin `parallel`, con poca ganancia adicional al pasar de 4 a 8 hilos. No se usó NumPy ni PyTorch en el código de implementación, según criterio docente. El tope de hilos de `numba.set_num_threads` en esta máquina es 8, no 10.

1. Introducción

La multiplicación de matrices es un núcleo costoso ($O(n^3)$) en el caso cúbico) y en Python el rendimiento depende de tres factores: el **orden** de bucles (acceso a columnas vs. filas transpuestas), el tipo de **paralelismo** (hilos vs. procesos) y la posibilidad de **compilar** el núcleo con Numba. El trabajo fija un entorno reproducible (mismas entradas y semilla) y exporta tiempos, *speed-up* y eficiencia vía el script de benchmark, para contrastar implementaciones y responder las preguntas de análisis de la consigna.

2. Metodología

2.1 Equipo

Propiedad	Valor
CPU	Intel(R) Core(TM) i5-10300H @ 2.50GHz (4 núcleos físicos, 8 lógicos)
Cores (lógicos)	8
RAM	15 GiB (orden de magnitud)
Python	3.14.3t (Free-Threaded)
GIL	Deshabilitado (build <i>free-threaded</i>)
OS	Linux (Manjaro, kernel 6.12)

No se reportan resultados con PyTorch/GPU: no se utilizaron en el código del trabajo según criterio actual del dictado. Archivos de resultados: `resultados_v2_c512.csv`, `resultados_v2_c1024.csv` (generados con `benchmark.py` en el directorio del TP2).

2.2 Algoritmos evaluados

1. **Secuencial (transpuesto)**: producto con filas de (B^T) (mejor localidad; baseline “rápido” conceptual).
2. **Secuencial (tradicional)**: triple bucle con acceso directo a $B[k][j]$.
3. **`concurrent.futures.ThreadPoolExecutor`**: cálculo de filas de (C) en paralelo.
4. **`threading manual`**: tareas análogas con cola o partición de filas.
5. **`concurrent.futures.ProcessPoolExecutor`**: bloques de filas en procesos separados.
6. **`numba (njit)`**: núcleo compilado, modo serial (`--no-parallel / 1 hilo`) o prange (`--parallel`) con `set_num_threads(workers)`.

Nota (workers 10 y Numba): en este equipo, Numba acepta como máximo **8** hilos en `set_num_threads`; las corridas paralelas con Numba usan **4 u 8**. Los demás métodos se midieron con la misma grilla 1, 4 y 8 donde aplica, alineada a los CSV reales.

2.3 Parámetros experimentales

Parámetro	Valores
(c) (complejidad, perfil cubo)	512, 1024
<i>workers</i>	1, 4, 8 (y secuenciales con <i>workers</i> 1)
Semilla	2026
Cota de enteros	<code>max_val = 256</code>

Parámetro	Valores
Perfil	cubo

2.4 Métricas

- **Tiempo (s):** fase A B (columna tiempo_segundos_ab del CSV; referencia para comparar algoritmos en las tablas siguientes).
- **Speed-up y eficiencia:** generados por el benchmark en el CSV (speed_up_ab, eficiencia_ab); el *speed-up* se define respecto del **secuencial tradicional** como referencia (valor 1.0 de eficiencia/speed-up en esa fila).
- **Checksum:** suma de todos los elementos de la matriz resultado; debe coincidir entre métodos y con la verificación vía $((A \ B)^T)$.
- **Performance (%):** en el sentido de la consigna, reportable como eficiencia $\times$ 100; valores $> 100\%$ o eficiencia > 1 surgen al comparar con un baseline distinto o al usar núcleos de forma más “efectiva” que el triple bucle lento, no como violación de límites físicos (ver discusión).

2.5 Link al repositorio

[Repositorio](#)

3. Resultados

Se resume la fase **A B**; el checksum es **-110463704** para (c=512) y **34818311** para (c=1024), con checksum_ab = checksum_btat en todas las filas.

3.1 c = 512

Algoritmo	c	p	Tiempo (s)	Speed-up	Eficiencia	Cores
secuencial (transpuesta)	512	1	11.662	1.15	1.15	8

Algoritmo	c	p	Tiempo (s)	Speed-up	Eficiencia	Cores
secuencial (tradicional)	512	1	13.425	1.00	1.00	8
concurrent.futures.ThreadPoolExecutor	512	1	11.530	1.16	1.16	8
concurrent.futures.ThreadPoolExecutor	512	4	3.631	3.70	0.92	8
concurrent.futures.ThreadPoolExecutor	512	8	3.375	3.98	0.50	8
threading	512	1	11.428	1.17	1.17	8
threading	512	4	3.557	3.77	0.94	8
threading	512	8	3.258	4.12	0.52	8
concurrent.futures.ProcessPoolExecutor	512	1	10.069	1.33	1.33	8
concurrent.futures	512	4	3.184	4.22	1.05	8

Algoritmo	c	p	Tiempo (s)	Speed-up	Eficiencia	Cores
utures .ProcessPoolExecutor						
concurrent.futures .ProcessPoolExecutor	512	8	3.071	4.37	0.55	8
numba (njit)	512	1	1.162	11.55	11.55	8
numba (njit)	512	4	1.105	12.15	3.04	8
numba (njit)	512	8	1.037	12.95	1.62	8

3.2 c = 1024

Algoritmo	c	p	Tiempo (s)	Speed-up	Eficiencia	Cores
secuencial (transpuesta)	1024	1	85.871	1.82	1.82	8
secuencial (tradicional)	1024	1	155.927	1.00	1.00	8
concurrent.futures .ThreadPoolExecutor	1024	4	29.687	5.25	1.31	8

Algoritmo	c	p	Tiempo (s)	Speed-up	Eficiencia	Cores
concurrent.futures.ThreadPoolExecutor	1024	8	27.556	5.66	0.71	8
threading	1024	4	30.872	5.05	1.26	8
threading	1024	8	27.604	5.65	0.71	8
concurrent.futures.ProcessPoolExecutor	1024	4	24.895	6.26	1.57	8
concurrent.futures.ProcessPoolExecutor	1024	8	23.912	6.52	0.82	8
numba (njit)	1024	4	4.977	31.33	7.83	8
numba (njit)	1024	8	4.695	33.21	4.15	8

Mejores tiempos (A B): para ambos (c), el mínimo corresponde a **numba** (≈ 1.0 s con (c=512); ≈ 4.7 s con (c=1024)). Entre threading y ThreadPoolExecutor con (c=1024) y (p=8) los tiempos (~ 27.6 s) son **muy cercanos**; puede marcarse a ambos como mejores dentro de la familia “solo Python puro + hilos” según la consigna. Para documentar con rigor el modo **Numba serial** (parallel=False) en (c=1024), hace falta una fila adicional (corrida puntual con matrices_numba.py --workers 1 --no-parallel).

4. Discusión

4.1 threading y el techo de rendimiento

El cuerpo del producto interno en **Python puro** es caro: interpretación, objetos, granularidad por fila. Aun con GIL deshabilitado, el *speed-up* no suele ser lineal en (p). Comparado con el secuencial **transpuesto** (ya mejora fuerte al tradicional), el margen adicional de los pools es acotado.

4.2 Transpuesta vs. tradicional

El cálculo $(C_{ij} = A_{i:} (B^T)_{:j})$ mejora la **localidad** respecto de indexar columnas de (B). Con (c=1024), el transpuesto (~86 s) frente al tradicional (~156 s) lo muestra de forma nítida: menos fallos de caché y acceso más predecible.

4.3 Multiprocessing y linealidad del *speed-up*

No es lineal en general: *pickling*, arranque de procesos, sincronización y contención de memoria añaden coste fijo. En el CSV la **eficiencia** con (p=4) puede superar 1.0 *respecto del baseline tradicional* mientras el código paralelo se parece más al camino “transpuesto” que al triple bucle lento; con (p=8), la eficiencia baja (más cerca del modelo de Amdahl y saturación de recursos en 4 nucleares físicos).

4.4 Numba njit incluso con parallel=False

Compila a **LLVM**, elimina el bucle intérprete en el núcleo y tipa enteros: el salto a ~1.2 s en (c=512) con un hilo frente a ~13 s del tradicional es principalmente el **JIT + código nativo**, no el paralelismo.

4.5 Eficiencia por encima del 100%

Sí, como **artefacto de la métrica** cuando *speed-up* y eficiencia se miden frente a un **baseline** (tradicional) que deja a otros algoritmos con “más de 100%” de *performance* relativa, o eficiencia > 1 en la escala del script. No implica >100% de uso de un único core ni viola límites físicos; es una convención de reporte. Para un análisis más intuitivo, puede citarse además el *speed-up* frente al secuencial **transpuesto**.

5. Conclusión

1. El **método transpuesto** en secuencial reduce fuerte el tiempo frente al triple bucle clásico en el rango de (c) medido.
2. **Hilos** (ThreadPoolExecutor, threading) ofrecen mejoras acotadas y tiempos similares entre sí; con (c=1024) y (p=8) resultados de ambos quedan **próximos** y pueden citarse en conjunto.
3. **Procesos** escalan de forma razonable pero con overhead distinto; en estos datos, ProcessPoolExecutor a veces mejora ligeramente a los pools de hilos a igual (p).
4. **Numba** aporta la mayor aceleración (por compilación), con ganancia adicional moderada al usar prange y más hilos.
5. Las **limitaciones** de este estudio: una sola máquina, tope de **8** hilos para Numba, métricas ancladas al tradicional; trabajos futuros podrían fijar un único baseline explícito en el análisis escrito o extender la grilla a otras (c).